10PEARLS SHINE INTERNSHIP PROGRAM

Air Quality Index Predictor

A 100% serverless, end-to-end machine learning system that forecasts the Air Quality Index for Karachi three days ahead — hourly data collection, a feature store, automated model selection across eight candidate families, a versioned model registry, SHAP explanations, hazardous-air alerts, and a live dashboard.

Author: Subhan Khan

Live dashboard: karachi-air-quality-index-predictor.streamlit.app

Repository: github.com/githubSubhanKhan/Air-Quality-Index-Predictor

Report date: 29 August 2026

Contents

1. Executive summary

This report documents the Air Quality Index (AQI) Predictor built for the 10Pearls Shine Internship Program. The brief (*AQI_predict-1.pdf*) asked for four pipeline stages — feature generation, historical backfill, model training, and a web app — automated end to end and backed by a feature store and a model registry, plus guideline items covering EDA, a variety of forecasting models, explainability, and hazardous-air alerts.

All four pipeline stages and all four guideline items are implemented and running in production. The system fetches live weather and pollutant data hourly, engineers features shared identically between training and serving, stores them in a MongoDB feature store, retrains a slate of eight candidate models daily with automatic winner selection per forecast horizon, versions every model in a registry with checksummed artifacts, and serves three-day forecasts through a public Streamlit dashboard with SHAP explanations and hazardous-air email alerts.

DATA COLLECTED	DAY 1 ACCURACY	DAY 2 ACCURACY	DAY 3 ACCURACY
8,723	**MAE 3.68**	**MAE 6.87**	**MAE 9.02**
hourly readings, 2025-08-15 → present	R^2 0.868	R^2 0.588	R^2 0.320

Of the 26 individual requirement rows in the brief, 24 are fully implemented and verifiable in the live system. The remaining two both stem from a single gap — deep-learning candidates (TensorFlow/PyTorch) have not yet been added to the model slate, and the FastAPI service, while built and tested, is not separately deployed since the dashboard reads the data store directly. Section 9 addresses this directly rather than glossing over it.

2. Requirement coverage

Every row below maps a requirement from the project brief to the part of the system that satisfies it. File and workflow names link to the source in the repository.

Step 1 — Feature pipeline

Requirement	Status	Implementation
Fetch raw weather + pollutant data from an external API	**Done**	`fetch_openweather.py` (pollutants, current + history, geocoding), `fetch_openmeteo.py` (historical weather), `fetch_waqi.py` (alternate live source)
Compute features (model inputs) and targets (model outputs)	**Done**	`build_openweather_features.py`, `feature_engineering.py`; targets in `train.py` (`add_forecast_targets`)
Store the features in a feature store	**Done**	`feature_store.py` — MongoDB Atlas collection `aqi_features`, upserted on `(city, timestamp)`
Time-based features (hour, day, month)	**Done**	`create_calendar_features` — hour, day, month, day-of-week, plus cyclical `hour_sin/cos`, `doy_sin/cos`
Derived features like AQI change rate	**Done**	`aqi_change_rate`, plus lags, rolling means/std, and deviation-from-trailing-mean features (§5.4)

Step 2 — Backfill historical (features, targets)

Requirement	Status	Implementation
Run the feature script over a range of past dates to build training data	**Done**	`backfill.py` — `python -m app.src.features.backfill --city karachi --days 365`. 365 days backfilled, pollutants from OpenWeather + weather from Open-Meteo

Step 3 — Training pipeline

Requirement	Status	Implementation
Fetch historical (features, targets) from the feature store	**Done**	`load_feature_store` in `train.py`
Train and evaluate the best model possible for this data	**Done**	Eight-candidate slate evaluated per horizon on every retrain; winner chosen on a held-out validation block
Experiment with Scikit-learn models (Random Forest, Ridge Regression)	**Done**	`random_forest`, `ridge`, `hist_gbm` in `candidates.py`; five-model notebook comparison in `lag_feature_engineering.ipynb`
Store the trained model in a Model Registry	**Done**	`model_registry.py` — MongoDB + GridFS, versioning, stages, promotion gate, rollback, retention

Evaluate with RMSE, MAE and R^2	**Done**	All three recorded per horizon, plus a persistence baseline R^2 and a skill score, with every registry version

Step 4 — Automate pipeline runs

Requirement	Status	Implementation
CI/CD running the feature script every hour	**Done**	`data_pipeline.yml` — GitHub Actions, `cron: "0 * * * *"`
CI/CD running the training script every day	**Done**	`daily_training.yml` — `cron: "30 2 * * *"`, with a post-retrain smoke test and candidate comparison rendered into the job summary

Step 5 — Web app

Requirement	Status	Implementation
Load the model and features from the feature store	**Done**	`predictor.py` loads the production stage of the registry; `forecast.py` loads features
Compute predictions and show them on a simple, descriptive dashboard	**Done**	`streamlit_app.py` — deployed and live
Use Streamlit/Gradio and Flask/FastAPI	**Done**	Streamlit dashboard + a FastAPI service with 10 endpoints (§6). The dashboard reads MongoDB directly so it can be deployed standalone

Guidelines

Guideline	Status	Implementation
Perform EDA to identify trends	**Done**	`eda.ipynb` — trends over time, distribution, hour-of-day and day-of-week profiles, missingness, correlation heatmap
A variety of forecasting models, from statistical modelling to deep learning	**Partial**	Statistical (seasonal naive, Holt-Winters ETS, seasonal AR / SARIMA) through tree ensembles and linear models are implemented in `statistical.py` and `candidates.py`; a deep-learning candidate is not yet added (§9)
Use SHAP or LIME for feature importance	**Done**	`explainer.py` — TreeSHAP + exact linear attribution, local and global, stored per registry version
Add alerts for hazardous AQI levels	**Done**	Dashboard banner + email alerts with health guidance (§5.9)

Final submissions

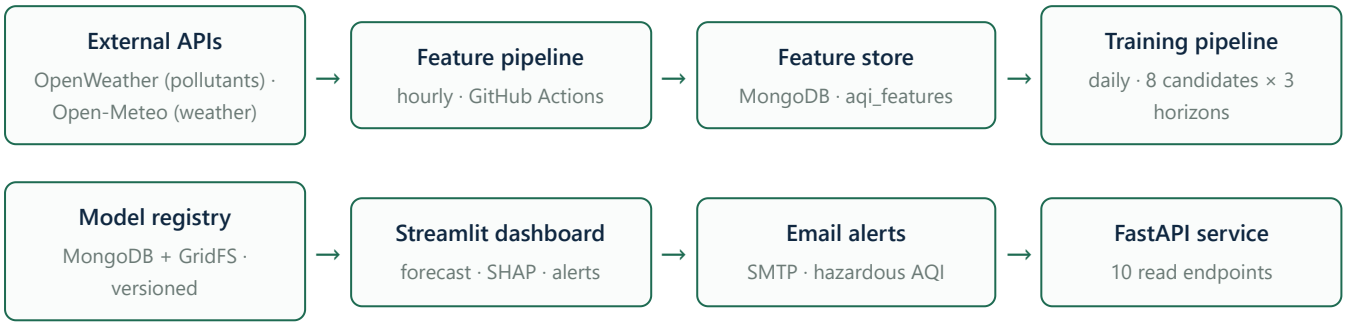
Deliverable	Status	Where
1. End-to-end AQI prediction system	**Done**	This repository
2. A scalable, automated pipeline	**Done**	Two GitHub Actions workflows; MongoDB + GridFS; no server to run

3. An interactive dashboard showing real-time and forecasted AQI	**Done**	Live Streamlit dashboard
4. A detailed report documenting everything achieved	**Done**	This document, and the project README

> 24 of the 26 requirement rows above are fully complete. The two open items both trace back to a single gap — a deep-learning candidate has not yet been added to the model slate — which is stated again, plainly, in Section 9 rather than left implicit.

3. System architecture

The system is serverless throughout — nothing needs to be kept running. GitHub Actions provides the scheduled compute, MongoDB Atlas holds all state, and Streamlit Community Cloud serves the front end, all on free tiers.



The forecast model

The forecast itself is persistence plus a damped, learned correction:

```
forecast = current AQI + alpha × model(features)
```

The model learns the *deviation* from the current reading rather than the absolute level, and `alpha` is fitted on a validation window. A model with no genuine skill collapses toward persistence instead of toward something worse — the change that fixed a previously negative day-3 R^2 (Section 7).

4. Results

Currently in production

Read live from the model registry, measured on a purged, held-out test block of 1,551 rows:

Horizon	Model	MAE	RMSE	R^2	Persistence R^2	Skill vs. persistence	alpha
Day 1 (1–24 h)	`random_forest` v5	**3.68**	4.70	**+0.868**	+0.866	+0.012	0.50
Day 2 (25–48 h)	`random_forest` v5	**6.87**	8.59	**+0.588**	+0.562	+0.060	0.50
Day 3 (49–72 h)	`hist_gbm` v5	**9.02**	11.04	**+0.320**	+0.255	+0.087	0.33

Each horizon is served by whichever candidate won it — the day-3 slot is a different model family from days 1 and 2.

What automated model selection bought

Versions 1–4 trained XGBoost only. Version 5 introduced per-retrain selection across the full candidate slate:

Horizon	v4 (XGBoost only)	v5 (selected)	Change
Day 1	MAE 3.84 · R^2 0.857 · skill −0.025	MAE 3.68 · R^2 0.868 · skill +0.012	Skill crossed from negative to positive
Day 2	MAE 7.29 · R^2 0.550	MAE 6.87 · R^2 0.588	−6% MAE
Day 3	MAE 9.51 · R^2 0.258	MAE 9.02 · R^2 0.320	−5% MAE, +0.06 R^2

At v4, day 1 was very slightly worse than doing nothing — a negative skill score against simply carrying the current reading forward. All three horizons are now positive.

The candidate slate, day 1

From a full-slate evaluation on 7,749 usable rows, ranked on validation MAE — the only column selection is allowed to see:

Candidate	Family	Features	Val MAE	Test MAE	Test R^2	Skill
`random_forest` — selected	tree	14	**6.76**	3.71	+0.866	+0.007
`seasonal_ar`	statistical	168	7.10	**3.59**	**+0.872**	**+0.052**
`hist_gbm`	tree	14	7.13	3.76	+0.862	−0.021
`xgboost`	tree	14	7.13	3.76	+0.861	−0.025
`persistence`	constant	—	7.27	3.71	+0.865	0.000
`holt_winters`	statistical	168	7.28	3.68	+0.867	+0.019
`seasonal_naive`	seasonal	168	7.31	3.71	+0.865	+0.005
`ridge`	linear	14	7.35	3.85	+0.857	−0.059

> Two things are worth reading off this table rather than glossing over. First, `seasonal_ar` has the best *test* MAE of all eight candidates, but ranked second on validation, so it was not chosen — selecting on the test column would be exactly the peeking the evaluation protocol exists to prevent. Second, at 24 hours nothing has much of an edge: six of eight candidates land within 0.26 MAE of each other, and persistence is among them. The models earn their keep at 48–72 hours, where skill rises to +0.06 and +0.09.

Historical notebook comparison (24 h-ahead, absolute AQI)

The original five-model bake-off in `lag_feature_engineering.ipynb`, kept for the record. This targets absolute AQI on 26 features — a different, harder target — so these numbers are not directly comparable with the tables above.

Model	MAE	RMSE	R^2
XGBoost	**8.88**	**11.85**	**0.464**
LightGBM	9.01	12.17	0.435
Random Forest	9.63	12.67	0.389
Ridge Regression	13.28	16.94	−0.093
Linear Regression	13.28	16.94	−0.093

5. How it works

5.1 Data collection

Karachi is geocoded once to latitude/longitude, then OpenWeather's Air Pollution API supplies PM2.5, PM10, O_3, NO_2, SO_2 and CO, and its Weather API supplies temperature, humidity, pressure and wind speed. OpenWeather returns concentrations rather than an index, so `aqi.py` converts PM2.5 to a standard US EPA AQI (0–500) using the official breakpoint table.

5.2 Feature store

MongoDB Atlas, collection `aqi_features`. Rows are upserted on `(city, timestamp)`, so re-running the pipeline or the backfill is idempotent rather than duplicating history. The same database holds the model registry, so the whole system needs exactly one connection string.

5.3 Backfill

`backfill.py` walks a date range, pulling pollutants from OpenWeather's history endpoint and weather from Open-Meteo's free archive, keyed by UNIX timestamp and merged per hour. 365 days of Karachi history was backfilled this way, which is what makes a seasonal model trainable at all.

5.4 Feature engineering

`feature_engineering.py` is shared by training and serving, so the two paths cannot skew apart. Three design choices matter most:

- **The hourly grid.** Backfilled rows sit exactly on the hour; live pipeline rows land at whatever minute the reading was taken. Timestamps are floored and reindexed onto a complete hourly grid, so `shift(24)` means "24 hours ago", not "24 rows ago". Gaps are forward-filled up to 6 hours, forward only, so a later reading never backfills an earlier hour.
- **Stationary features.** Karachi's pollutant levels drift substantially across a year (O_3 fell 54% between the training and evaluation windows), so the model set is built from relative features — deviations from trailing means — which stay in range at the far horizons. Raw day/month are replaced with cyclical sin/cos encodings so an untrained calendar branch is never hit.
- **The raw history block.** `aqi_hist_0` through `aqi_hist_167` (a week of hourly history), used only by the statistical forecasters that model the series directly rather than a feature-to-target mapping.

The 14 features the production models actually use:

```
aqi, aqi_roll_mean_24, aqi_roll_mean_168, aqi_rel_24, aqi_rel_168, aqi_trend_24_168,
aqi_roll_std_24, doy_sin, doy_cos, hour_sin, hour_cos, pm25_rel_168, humidity, wind_speed
```

5.5 Model training and selection

Every retrain trains a candidate slate per horizon and keeps whichever wins on a validation block it was not fitted on. The slate spans statistical, linear and tree-ensemble families, defined in `candidates.py`:

Candidate	Family	Estimator	Inputs

`persistence`	constant	`DummyRegressor` — the null model	14
`seasonal_naive`	seasonal	This hour tomorrow = this hour today	168
`holt_winters`	statistical	Additive Holt-Winters ETS, damped trend + 24 h seasonality	168
`seasonal_ar`	statistical	SARIMA(p,0,0)(0,1,0)[24] — seasonal difference + AR(p)	168
`ridge`	linear	`StandardScaler` + `Ridge`	14
`random_forest`	tree	`RandomForestRegressor`	14
`hist_gbm`	tree	`HistGradientBoostingRegressor`	14
`xgboost`	tree	`XGBRegressor` — the incumbent	14

Every candidate predicts the same quantity — the deviation of the horizon's mean AQI from the current reading — and is scored on the reconstructed AQI forecast, not its own internal output. Each gets its own `alpha`, fitted on validation, so a family whose predictions are mostly noise is damped toward persistence automatically. A 2% margin guards the incumbent: a challenger must beat XGBoost by 2% on validation MAE to take the slot, so the served family does not flip between daily retrains on differences inside the noise. Every candidate's metrics are stored with the version, so the comparison behind a served model survives the run that produced it.

The statistical models

`statistical.py` implements the classical end on numpy/scipy — no heavy new dependency. Parameters are fitted once on roughly 33 week-long training segments; state is rebuilt per row from that row's own trailing week, so training and serving share one code path. Order selection for the seasonal AR model rejects non-stationary fits — least squares will happily return a root on the unit circle, and forecasting 72 steps forward with that would diverge exponentially. The chosen AR(24) has a largest root of 0.998: stationary, but close enough to the boundary that skipping the check would be relying on luck as the data store grows.

5.6 Model registry

Trained models are not loose `.pkl` files. `model_registry.py` gives every published model an immutable, monotonically increasing version per model name; the metrics it was evaluated with and the full candidate comparison it won; its hyperparameters, feature list, data lineage and library versions; a lifecycle stage (staging, production, archived); and a SHA-256-checksummed artifact in GridFS, verified on load. Promoting or rolling back a model is a metadata change — no redeploy, no code change. A promotion gate refuses a candidate more than 25% worse than the incumbent on MAE, and retention prunes old binaries while keeping metric history forever.

5.7 Forecasting and serving

`predictor.py` loads the production version of each horizon, caching artifacts for the process lifetime but re-checking promoted version numbers every 15 minutes, so a promotion or rollback is picked up without restarting anything. Each horizon carries its own feature list and its own target transform, so one horizon can be retrained independently of the other two.

5.8 Explainability

`explainer.py` produces a local view — why this specific forecast came out where it did, with each feature's signed contribution in AQI points such that `base_value + Σ contributions` reconstructs the prediction exactly — and a global view — mean absolute SHAP value per feature over the evaluation rows, stored with the registry version. The attribution method is chosen from what the fitted estimator exposes: TreeSHAP for tree ensembles, exact coefficient-based attribution for linear models, and zero attribution against a defined baseline for series/constant models, so every model family explains itself correctly regardless of which one wins a given retrain.

5.9 Hazardous air alerts

The six EPA AQI categories, their bounds and their health advice live in `aqi.py`, shared by the dashboard and the alert emails so the two can never disagree. On the dashboard, a sidebar control sets the alert threshold, defaulting to *Unhealthy for Sensitive Groups* (101+); when any of the three forecast days reaches it, a banner names the day, the AQI and what to do. By email, an "Email This Forecast" section sends the full three-day outlook with health guidance, built on the standard library's `smtplib` (`messages.py`, `mailer.py`). Every send reports all three days regardless of severity — mail that only arrives on bad days is indistinguishable from a mail system that has quietly broken — and sends are rate-limited (5 per session, 45-second cooldown) since the dashboard is public.

5.10 Automation

Workflow	Schedule	What it does
`data_pipeline.yml`	Hourly (`0 * * * *`)	Fetches the current reading and upserts it into the feature store
`daily_training.yml`	Daily (`30 2 * * *`)	Retrains all 8 candidates × 3 horizons, publishes winners to the registry, applies the promotion gate, prunes old artifacts, smoke-tests that production still serves a forecast, and renders the candidate comparison into the job summary

Both workflows take credentials from GitHub Secrets. Nothing is committed back to the repository — retrained models go to the registry, and serving reads the production stage from there.

6. API reference

A FastAPI service (`app/main.py`) exposes the same data read-only, run locally via `uvicorn app.main:app --reload` with interactive docs at `/docs` .

Method	Endpoint	Purpose
GET	`/`	Service banner
GET	`/health/`	Health check
GET	`/cities`	Every city present in the feature store
GET	`/predict/{city}`	Current AQI + the 3-day forecast
GET	`/history/{city}?hours=168`	Recent feature-store readings for charting
GET	`/models/`	The versions currently serving, with their metrics
GET	`/models/versions?name=&limit=`	Registry version history, newest first
GET	`/models/names`	Every model name in the registry
GET	`/explain/{city}?horizon=&top=`	SHAP contributions behind the current forecast
GET	`/explain/global`	Global mean |SHAP| for the production versions

The API is read-only by design: promotion and rollback change what production serves, so those actions stay in the registry command-line tool rather than on an unauthenticated endpoint.

7. Evaluation methodology

The reasoning behind why the numbers in Section 4 can be trusted.

- **Chronological, nested, purged split.** Rows are ordered in time and split 65% train / 15% validation / 20% test, never shuffled. Because a row's target window reaches 72 hours ahead, 72 rows are dropped on either side of each boundary — without that purge, training targets would overlap the test window and the reported metrics would be optimistic.
- **Validation ranks; test is opened once.** Training fits each candidate, validation fits its `alpha` and decides the winner, and the test block is scored at the end and never consulted before that decision — which is why `seasonal_ar` tops the day-1 test column and still was not selected.
- **A baseline that is hard to beat.** Every horizon reports the persistence baseline's R^2 on the same window and a skill score against it, positive only when the model genuinely beats simply carrying the current reading forward. Day 1 has R^2 0.868 but skill of only +0.012 — the report states this rather than quoting R^2 alone.
- **Correction-weight shrinkage.** `alpha` is fitted on roughly 1,000 autocorrelated rows, so the raw least-squares estimate is noisy; it is halved before use, a rule that outperformed the unshrunk fit in 11 of 12 window × horizon combinations tested.
- **Reproducibility.** Every registry version records the library versions it was trained with, its exact feature list, hyperparameters, training data row counts and timestamp span, the commit SHA that produced it, and a SHA-256 checksum of its artifact, verified on load.
- **Verification.** The statistical forecasters are tested against series whose answer is known by construction — the seasonal AR model recovers a planted AR(1) coefficient of 0.600 as 0.604, and seasonal naive reproduces a known daily profile exactly. All eight candidates are checked for the SHAP reconstruction identity, alpha bounds and metadata serialisability. The alert system's SMTP conversation is verified against a fake server, covering STARTTLS ordering, multipart structure, the SSL path, auth-failure handling and address validation.

8. Tech stack

Layer	Choice	Why
Data sources	OpenWeather, Open-Meteo, WAQI	Free tiers with both current and historical coverage
Feature store	MongoDB Atlas	Free tier, and one connection string serves both the store and the registry
Model registry	MongoDB + GridFS	Versioning, stages and checksummed artifacts without hosting a second service
Modelling	scikit-learn, XGBoost, numpy/scipy	Tree ensembles, linear and classical statistical models under one estimator interface
Explainability	SHAP (TreeSHAP) + exact linear attribution	Local and global attribution that reconstructs the prediction exactly
Dashboard	Streamlit + Plotly	Fastest path to a deployable, interactive front end
API	FastAPI + Uvicorn	Typed routes and automatic OpenAPI docs
Alerts	`smtplib` (standard library)	No dependency, no third-party mail service
Automation	GitHub Actions	Free scheduled compute; no server to keep running

9. Honest limitations

Stated plainly, in keeping with how the rest of this report is written — a system whose gaps are documented is more trustworthy than one that claims to have none.

> **Deep-learning models are not yet in the candidate slate.** This is the one outstanding item from the "variety of forecasting models" guideline. The slate is designed to take one — a new candidate needs only a fit/predict estimator, a declared feature set and a family label — and an LSTM or small MLP over the existing 168-hour history block would drop in directly. Given that six of the eight current candidates already land within 0.26 MAE of each other on day 1, the realistic expectation is that it would be competitive rather than transformative on one year of single-city data.

> **The FastAPI service is built and tested but not separately deployed.** It runs locally and is exercised in development; the live, deployed dashboard reads MongoDB directly so it can be hosted standalone on Streamlit Community Cloud without a second running service.

Two smaller, already-mitigated issues worth naming: GitHub Actions' hourly cron is best-effort on low-activity repositories, so the forecast is occasionally anchored on a reading a few hours older than the newest one — this is surfaced on the dashboard rather than hidden, with the reading time shown on screen and in every alert email. And the system currently trains and serves one city, Karachi; every layer (store, registry, dashboard) is already parameterised by city, so adding a second is a workflow argument rather than a code change.